Strings in C

1. Introduction

A **string** in C is a **sequence of characters stored in an array** and terminated by a special character called the **null character (\0)**.

Strings are used to store **text data** such as:

- Names
- Addresses
- Messages
- Words or sentences

Example strings:

"Hello"

"Computer"

"Programming"

2. How Strings are Stored

In C, strings are stored as **character arrays**.

Example:

HELLO

Stored in memory as:

Index	**Value**
0	H
1	E
2	L
3	L
4	O
5	\0

The **\0 (null character)** marks the **end of the string**.

3. Declaring a String

Syntax

char string_name[size];

Example:

char name[20];

This creates a string that can hold **up to 19 characters + null character**.

4. Initializing a String

Method 1

char name[] = "John";

Method 2

char name[5] = {'J','o','h','n','\0'};

Both methods store the same string.

5. Reading and Displaying Strings

Using scanf()

```
#include <stdio.h>
int main()
{
   char name[20];

   printf("Enter your name: ");
   scanf("%s", name);

   printf("Hello %s", name);

   return 0;
}
```

Example Output

Enter your name: John

Hello John

⚠ **Important:**

scanf("%s") stops reading **when it encounters a space**.

Example input:

John Smith

Only **John** will be stored.

6. Using gets() and puts()

gets() reads a full line including spaces.

Example:

```
#include <stdio.h>

int main()
{
   char name[50];

   printf("Enter your full name: ");
   gets(name);

   puts(name);

   return 0;
}
```

Output

```
Enter your full name: John Smith

John Smith
```

7. String Functions

C provides built-in **string functions** in the library:

<string.h>

Common string functions include:

Function	Purpose
strlen()	Find string length
strcpy()	Copy one string to another
strcat()	Concatenate (join) strings
strcmp()	Compare two strings

1. strlen() – Find Length of String

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
   char name[] = "Computer";

   printf("Length = %d", strlen(name));

   return 0;
}
```

Output

Length = 8

2. strcpy() – Copy a String

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
   char source[] = "Hello";
   char destination[20];

   strcpy(destination, source);

   printf("%s", destination);

   return 0;
}
```

Output

Hello

3. strcat() – Join Two Strings

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str1[20] = "Hello ";
   char str2[] = "World";

   strcat(str1, str2);

   printf("%s", str1);

   return 0;
}
```

Output

Hello World

4. strcmp() – Compare Strings

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str1[] = "Apple";
   char str2[] = "Apple";

   if(strcmp(str1,str2) == 0)
      printf("Strings are equal");
   else
      printf("Strings are not equal");

   return 0;
}
```

Output

Strings are equal

8. Traversing a String

A string can be processed **character by character using a loop**.

Example:

```
#include <stdio.h>

int main()
{
   char name[] = "HELLO";

   for(int i = 0; name[i] != '\0'; i++)
   {
      printf("%c\n", name[i]);
   }

   return 0;
}
```

Output

```
H

E

L

L

O
```

9. Example: Count Characters in a String

```
#include <stdio.h>

int main()
{
   char text[100];
   int count = 0;

   printf("Enter text: ");
   gets(text);

   for(int i = 0; text[i] != '\0'; i++)
   {
      count++;
   }

   printf("Length = %d", count);

   return 0;
}
```